

Garbage collector not access the string pool/constant
 All the ~~java~~ SCP object will be destroyed automatically at the
 time of JVM shutdown.
 no duplicate object will be available in scp.

Chapter 6: String, Arrays and Command Line Argument

- String
- Arrays
- Command Line Arguments
- Variable Arguments

* we can create our own immutable class. ✓

String

- It is the most extensively class in Java.
- It can store maximum of 2 billion Unicode characters (256 KB)

Constructors

String()
 String(String)
 String(char c[], int startIndex, int num)
 String(byte b[], int startIndex, int num)

```
String s1 = new String();
String s2 = new String("Welcome");
char c[] = {'W','e','l','c','o','m','e'};
String s3 = new String(c, 3, 4);
byte b[] = {65, 66, 67, 68, 69, 70};
String s4 = new String(b, 0, 5);
```

String s5 = new String("Welcome"); → standard तरीका
 String s6 = "Welcome"; → बिना new का बिना ही डाल सकते है String (only in String)

Methods

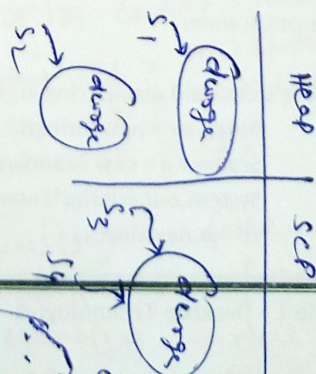
int indexOf(String str)
 int indexOf(String str, int startIndex)
 int lastIndexOf(String str)
 int lastIndexOf(String str, int startIndex)
 char charAt(int index)

```
String str = new String("Welcome");
```

```
int a = str.indexOf("e"); //1
int b = str.indexOf("e", a+1); //6
int c = str.lastIndexOf("e"); //6
int d = str.lastIndexOf("e", c-1); //1
int e = str.indexOf("me"); //5
int f = str.indexOf("zee"); //-1
char g = str.charAt(3); //c
char h = "Hello".charAt(4); //o
```

Anonymous String

W	e	l	c	o	m	e
0	1	2	3	4	5	6



String cannot be immutable

length and size counted from 1 but index starts from zero

SCP → Specially designed ^{memory} ~~area~~ for string object

vedisoft
Software & Education Services Pvt. Ltd.

String toUpperCase()

String toLowerCase()

boolean equals(String)

boolean equalsIgnoreCase(String)

boolean startsWith(String)

boolean endsWith(String)

→ insensitive

String s1 = new String("Hello World");

String s2 = s1.toUpperCase();

//s1 = Hello World, s2 = HELLO WORLD

boolean b = s1.equals(s2);

//false

boolean b1 = s1.equalsIgnoreCase(s2);

//true

boolean b2 = s1.startsWith("Hell");

//true

boolean b3 = s2.endsWith("ing");

//false

String trim() → leading and trailing space remove

int length() → return length of string

String substring(int stindex)

String substring(int stindex, int endindex)

String s1 = new String(" Hello World ");

int a = s1.length();

//16

String s2 = s1.trim();

int b = s2.length();

//12

String s3 = s2.substring(0,5);

//Hello

String s4 = s2.substring(7);

//World

starting mai 4th ending 1st mai

String[] split(String separator)

String concat(String str)

String replace(char original, char replacement)

String str = new String("This is a new world");

String s[] = str.split(" ");

→ case sensitive

String s1 = "one";

String s2 = s1.concat("two");

//onetwo

String s = "Hello".replace('l', 'w');

//Hewwo

→ मया string बन रहा है क्योंकि
पुराना string change हो
जाता है

Write a program that accepts an RGB value from user and print parts of red, green and blue.

import java.util.Scanner;

class Rgb {

public static void main(String args[]) {

String str = new String();

Scanner a = new Scanner(System.in);

System.out.println("Enter Rgb Value:");

str = a.nextLine();

student
class
in
green

equals method
efficient
to
string

String class overrides the equals() method of Object class. So you can compare the contents of two strings by equals() method.

String s[] = str.split(",");
System.out.println("Red is : " + s[0]);
System.out.println("Green is : " + s[1]);
System.out.println("Blue is : " + s[2]);

StringBuffer class doesn't override the equals() method of Object class

Write a program that accepts a string. The string should have at least 5 characters and should not contain any special characters or digits.

import java.util.Scanner;

class TestString {

public static void main(String args[]) {

String str = new String();

Scanner in = new Scanner(System.in);

main:

while(true) {

System.out.print("Enter String: ");

str = in.nextLine();

if(str.length() < 5) {

System.out.println("Error !! Atleast 5 character is required.");

continue;

}

for(int i = 0; i < str.length(); i++) {

char c = str.charAt(i);

if (!(c >= 'A' && c <= 'Z' || c >= 'a' && c <= 'z')) {

System.out.println("Error !! Special characters or digits found. ");

continue main; → label + ET → immediate loop ko continue.

}

}

break;

}

}

For primitive types
== can be used.

एक ही पंक्ति

Array Long size array is not allowed in Java.

Collection of similar data types that shares common name and memory.

Terms:

- Each data item in an array is called "element"
- The counting of elements is called "index" or "subscript"
- The total number of elements is called "length" or "size"
- Index always starts from 0.

Declaration and Initialization

<Data Type> <arrayVariable>[] = new <Data Type>[size];

we can override
equals() method for content
comparing.

s1.equals(s2) → false

s1.equals(s3) → false

s1.equals(s4) → true

String class equals method ko
override krta hai

Object class ko equals() ko
Reference check krta hai
or content check like
Student s1 = new Student("Arinash");
Student s2 = new Student("Arinash");
Student s4 = s1;

array में size initialize करने समय ध्यान दे

array के लिए int / float के लिए नहीं

Example:

```
int num[] = new int[10];
float f[] = new float[5];
```

Declaration

```
int[] num, a;
float[] f;
```

```
int num[];
float f[];
```

```
int []num;
float []f;
```

multiple

Initialization

```
num = new int[10];
```

```
f = new float[5];
```

The length property

This property returns length of an array

```
num.length // 10;
```

```
f.length // 5;
```

→ left hand side में size नहीं बता सकते हैं

```
int num[][] = new int[4][5];
```

```
String wdays[] = { "Sunday", "Monday", "Saturday" };
```

```
int n[] = { 1, 2, 3, 4, 5};
```

```
int num[][] = {
```

```
    {1,2,3,4,5},
```

```
    {1,2,3,4,5},
```

```
    {1,2,3,4,5}
```

```
};
```

always use length property in java

Write a program that accepts 10 numbers in an array and display their sum and average.

```
import java.util.Scanner;
```

```
class Array1 {
```

```
    public static void main(String args[]) {
```

```
        int num[] = new int[10];
```

```
        int i, sum = 0, avg = 0;
```

```
        Scanner in = new Scanner(System.in);
```

```
        for(i = 0; i < num.length; i++) {
```

```
            System.out.print("Enter Number " + (i + 1) + " : ");
```

```
            num[i] = in.nextInt();
```

```
            sum += num[i];
```

```
        }
```

```
        avg = sum / num.length;
```

```
        System.out.println("Sum : " + sum);
```

```
        System.out.println("Average : " + avg);
```

```
    }
```

```
}
```

प्रोग्राम की मदद के साथ किया सकते हैं

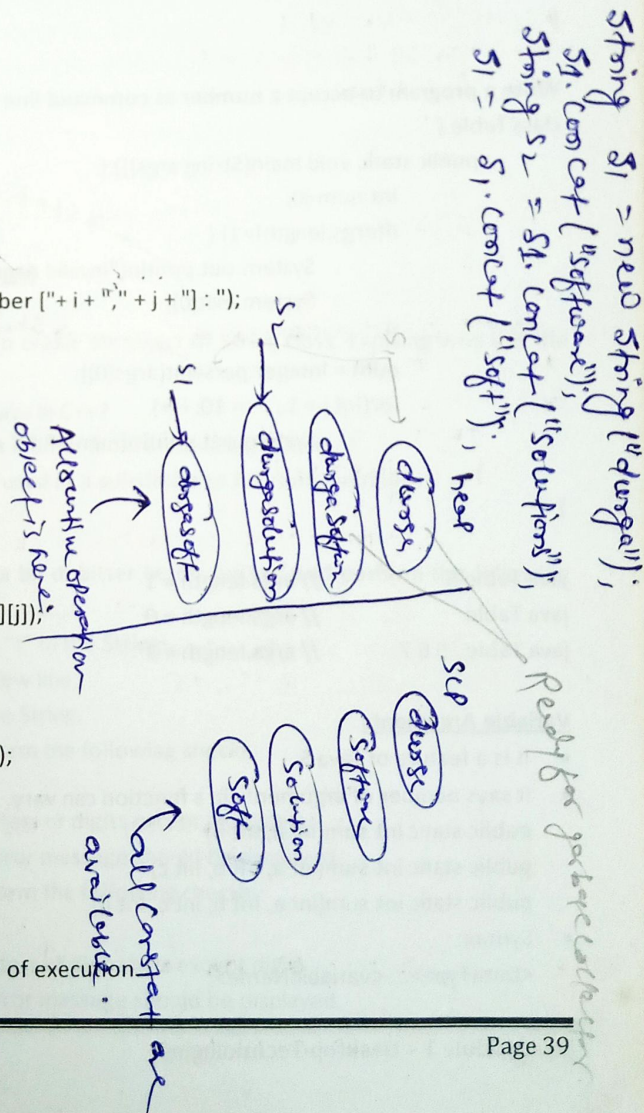
Write a program that initializes an array by multiples of 5 from 100 to 200 and display them.

in Java Array Size can be put during the runtime

```
class Array2 {
    public static void main(String[] args) {
        int num[], j, i;
        for(i = 100, j = 1; i < 200; i += 5, j++);
        num = new int[j]; // initialization
        for(i = 100, j = 0; i < 200; i += 5, j++)
            num[j] = i;
        for(j = 0; j < num.length; j++)
            System.out.println(num[j]);
    }
}
```

Write a program that creates matrix of size 4 by 4. Accepts all integer elements and print sum of all rows, all columns and entire matrix.

```
import java.util.Scanner;
class Array3 {
    public static void main(String args[]) {
        int num[][] = new int[4][4];
        int rowSum[] = new int[num.length];
        int columnSum[] = new int[num[0].length];
        int i, j, sum = 0;
        Scanner in = new Scanner(System.in);
        for(i = 0; i < num.length; i++) {
            for(j = 0; j < num[i].length; j++) {
                System.out.print("Enter Number [" + i + " , " + j + " ] : ");
                num[i][j] = in.nextInt();
                sum += num[i][j];
                rowSum[i] += num[i][j];
                columnSum[j] += num[i][j];
            }
        }
        for(i = 0; i < num.length; i++) {
            for(j = 0; j < num[i].length; j++)
                System.out.print("\t" + num[i][j]);
            System.out.println("\t" + rowSum[i]);
        }
        for(j = 0; j < num[0].length; j++)
            System.out.print("\t" + columnSum[j]);
        System.out.print("\t" + sum);
    }
}
```



Command Line Arguments

- These are details that are supplied to program at the time of execution.

- These details gets stored inside String args[]

Example:

```
java Program One Two Three
args[0]=One
args[1]=Two
args[2]=Three
args.length=3
```

Objective: It is used to take short input during execution

Write a program to displays all the command line arguments

```
class Command1 {
    public static void main(String args[]) {
        for(int i = 0 ; i < args.length ; i++)
            System.out.println(args[i]);
    }
}
```

Write a program to accept a number as command line argument and display its table

```
class Table {
    public static void main(String args[]) {
        int num=0;
        if(args.length!=1) {
            System.out.println("Invalid Arguments");
            System.exit(0);
        }
        num = Integer.parseInt(args[0]);
        for(int i = 1 ; i <= 10; i++)
            System.out.println(num + " * " + i + " = " + num*i);
    }
}
```

static function of class System

String to integer A to S, D, E, I of class DNT.

```
java Table 5          // args.length = 1
java Table            // args.length = 0
java Table 5 6 7      // args.length = 3
```

Variable Arguments

- It is a feature of Java 5
- It says number of arguments in a function can vary.

```
public static int sum(int a, int b)
public static int sum(int a, int b, int c)
public static int sum(int a, int b, int c, int d)
```

- Syntax:

```
<DataType>... <variableName>;
```

Static function → only work on static data
for Reusability)

Example:

int... num;

- Internally variable argument is treated as an array (superset)

Array के static variable argument
sim sikhte hai

Rules:

- There can be only 1 variable argument in a function.
- This should always be the last argument in the argument list.

Example

class VarArgs {

```
public static int sum(int... n) {  
    int sum = 0;  
    for(int i = 0; i < n.length; i++)  
        sum += n[i];  
    return sum;  
}
```

for Reusability (remember date par kaha nahi karata hai)

isliye hume kahi declare kar sakte hai par code me nahi

par array variable Argument ke static
par array nahi sim sikhte hai

```
public static void main(String... args) {  
    System.out.println(sum(10, 20, 30));  
    System.out.println(sum());  
    System.out.println(sum(10, 20, 30, 40, 50));  
    int num[] = {9, 9, 9};  
    System.out.println(sum(num));  
}
```

Assignment

Theory Questions

- What is the range of String class? Write different ways to create an object of String class. Explaining why it is the most extensively used class in Java.
- What is an array? How does arrays in Java differ from arrays in C++?
- Why do we need command line arguments and how does it differ from input using Scanner?
- What are variable arguments? Can variable argument be used as a substitute to function overload.

Practical Questions

- Write a program that initializes a String with "Betty got a bit of bitter better butter" and perform the following operations:
 - Display all the occurrences (index) of "b" and "t" in the String.
 - Display each word along with its length in a new line.
 - Display all the words that starts with "b" in the String.
- Write a program that accepts a String from user and perform the following checks:
 - The string should have atleast 3 characters.
 - The string should not have any special characters or digits except alphabets.
If any condition is not satisfied appropriate error message should be displayed.
- Write a program that accepts a String from user and perform the following checks:
 - The string should have atleast 5 characters.
 - The string should not have any special characters or alphabets except digits.
If any condition is not satisfied appropriate error message should be displayed.

Chapter 7: enum and interface

- enum
- interface

enum

- an enumeration is a list of named constants
- enum was introduced with jdk5
- enum is used to store one out of multiple pre-defined set of values
- for each enum, internally Java creates a class file
- each constant declared within enum is declared as public static final object of the class within the class created

Example: → default & public are valid access modifiers.

```
public enum Colors { RED, GREEN, BLUE };
class Test1 {
    public static void main(String args[]) {
        Colors color = Colors.RED;
        System.out.println(color);
    }
}
```

on compilation .class
class Colors {
public static final Colors RED = new Colors();
GREEN = new Colors();
BLUE = new Colors();
}

- How to declare and use enum
- enum can be declared within class or outside class
- if enum is declared outside class and is public, enum name and filename should be same
- enum makes use of proper case

Example: in one class public enum or public class होता है तो नमो public होता है।

```
class Test2 {
    enum Colors {
        RED("#FF0000"), GREEN("#00FF00"), BLUE("#0000FF");
        Colors(String hexCode) {
            this.hexCode = hexCode;
        }
        public String getHexCode() {
            return hexCode;
        }
        private String hexCode;
    }
    public static void main(String args[]) {
        Colors color = Colors.RED;
        System.out.println(color);
        System.out.println(color.getHexCode());
    }
}
```

here private, default, protected & public and static also are valid access modifiers
→ enum के साथ data bind करती है

↑ constructor

← hexcode को store कर रहे हैं String में

- enum can have data, constructors and methods
- options declared within enum should be declared within the first line

- enum cannot have public constructors

Example:

```
class Test3 {  
    enum Colors { RED, GREEN, BLUE }  
    public static void main(String args[]) {  
        for(Colors c : Colors.values())  
            System.out.println(c + " : " + c.ordinal());  
  
        System.out.println(Colors.valueOf("RED"));  
        System.out.println(Colors.valueOf("YELLOW"));  
    }  
}
```

- valueOf(), values() and ordinal()

Example:

```
class Test4 {  
    enum Colors {RED, GREEN, BLUE};  
    public static void main(String args[]) {  
        Colors color = Colors.RED;  
        if(color == Colors.RED)  
            System.out.println("This is Red Color.");  
        else  
            System.out.println("This is not the Red Color.");  
  
        if(color.equals(Colors.RED))  
            System.out.println("This is Red Color.");  
        else  
            System.out.println("This is not the Red Color.");  
  
        switch(color) {  
            case RED : System.out.println("This is Red Color.");  
                        break;  
            case GREEN : System.out.println("This is Green Color.");  
                        break;  
            case BLUE : System.out.println("This is Blue Color.");  
                        break;  
        }  
    }  
}
```

- equals(), ==, switch works with enum

Example:

```
import java.util.Scanner;
```

```
enum Gender {
    Male, Female;
    public static Gender acceptGender() {
        Scanner in = new Scanner(System.in);
        String str = in.nextLine();
        if(str.trim().toLowerCase().equals("male"))
            return Male;
        else
            return Female;
    }
};
```

```
class Person {
    protected String name;
    protected int age;
    protected Gender gender;
    public Person() {
        name = new String();
        gender = Gender.Male;
    }
    public Person(String name, int age, Gender gender) {
        this.name = name;
        this.age = age;
        this.gender = gender;
    }
    public void acceptData() {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter Name : ");
        name = in.nextLine();
        System.out.println("Enter Age : ");
        age = in.nextInt();
        System.out.println("Enter Gender : ");
        gender = Gender.acceptGender();
    }
    public void showData() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Gender : " + gender);
    }
    public static void main(String args[]) {
        Person p = new Person();
        p.acceptData();
        p.showData();
    }
}
```

here ABC class can access point class
bcoz point is default & point of ABC are
have same Direction

class ABC

```
{
    public static void main(String args[]) {
        Point p = new Point();
        p.acceptData();
        p.showData();
    }
}
```

An interface does not contain constructor bcoz every data in interface is by default - public, static, final
 → 100% Pure abstract class.
 → no instance variable

Interface

- Interface is a collection of method declarations/prototype
- If any class implements an interface, it provides definition for all the methods declared within interface.
- A class can implement, one or more interfaces.
- Apart from interface methods class can also have its own method.
- An interface can inherit one or more interfaces.
- By default all methods declared within interface are public and abstract.
- By default all data declared within interface is public, static and final.

```
interface Ia {
    void a();
    int x = 10;
}
```

final (constant)

```
//public abstract void a();
//public static final int x = 10;
```

```
class Ca implements Ia {
    public void a(){....}
}
```

* I can replace every interface with an abstract class but this is not a good programming practice

```
interface Ib {
    void b();
}
```

```
class Cab implements Ia, Ib {
    public void a(){...}
    public void b(){...}
    public void c(){...}
}
```

हो सकता है (सबूत का)

NOTE:

- Interfaces make use of ProperCase
- Interface name and filename should be same

Example: EmpInterface.java

```
interface EmpInterface {
    void showData();
    void acceptData();
}
```

Example: Employee.java

```
import java.util.Scanner;
class Employee implements EmpInterface
    protected int code;
    protected String name;
    protected String dept;
    protected String desig;
```

Compile तब तक होता है जब तक interface prototype / Declaration नहीं होता

Any Service Requirement Specification is interface
eg. JDBC API, Servlet API

protected float basic;

public Employee() {

name = new String();

dept = new String();

desig = new String();

}

public void acceptData() {

Scanner a = new Scanner(System.in);

System.out.println("Enter Code : ");

code = a.nextInt();

a.nextLine();

System.out.println("Enter name : ");

name = a.nextLine();

System.out.println("Enter Department : ");

dept = a.nextLine();

System.out.println("Enter Designation : ");

desig = a.nextLine();

System.out.println("Enter Basic : ");

basic = a.nextFloat();

}

public void showData() {

System.out.println("Code: " + code);

System.out.println("Name: " + name);

System.out.println("Department: " + dept);

System.out.println("Designation: " + desig);

System.out.println("Basic: " + basic);

}

public float getHra() {

return 0.2f * basic;

}

public static void main(String args[])

Employee e = new Employee();

e.getData();

e.showData();

System.out.println("HRA is : " + e.getHra());

EmpInterface e1 = new Employee();

e1.getData();

e1.showData();

// WRONG e1.getHra();

}

Default Interface Methods

- The release of JDK 8 has changed this by adding a new capability to interface called the default method (also referred as extension method).

- A default method lets you define a default implementation for an interface method. In other words, by use of a default method, it is now possible for an interface method to provide a body, rather than being abstract.

```
interface Stack {
    void push(int element);
    void pop();
    default void clear() {
        System.out.println("Clear Implementation not provided");
    }
}
```

Handwritten notes:
 - "definition" with an arrow pointing to the method signatures.
 - "it's compulsory" with an arrow pointing to the curly braces of the interface.
 - "Optional" with an arrow pointing to the default method.
 - "definition" with an arrow pointing to the body of the default method.

```
public class StackImpl implements Stack {
    private static final int capacity = 3;
    int arr[] = new int[capacity];
    int top = -1;
    public void push(int pushedElement) {
        if (top < capacity - 1) {
            top++;
            arr[top] = pushedElement;
            System.out.println("Element " + pushedElement + " is pushed to Stack !");
            printElements();
        } else {
            System.out.println("Stack Overflow !");
        }
    }
    public void pop() {
        if (top >= 0) {
            top--;
            System.out.println("Pop operation done !");
        } else {
            System.out.println("Stack Underflow !");
        }
    }
    public void printElements() {
        if (top >= 0) {
            System.out.println("Elements in stack :");
            for (int i = 0; i <= top; i++) {
                System.out.println(arr[i]);
            }
        }
    }
    // public void clear() {
    //     arr = new int[capacity];
    //     top = -1;
    // }
```

```
// }
public static void main(String[] args) {
    StackImpl stackDemo = new StackImpl();
    stackDemo.pop();
    stackDemo.push(23);
    stackDemo.push(2);
    stackDemo.push(73);
    stackDemo.push(21);
    stackDemo.pop();
    stackDemo.pop();
    stackDemo.pop();
    stackDemo.pop();
}
```

	Declaration	Definition	method inside
	Java	Programmer	Java
Real Program	programmer	programmer	programmer
This programmer			

from client point the set of services what is expecting is interfaces, from service provider's point of view the set of service he is offering is interface,
so any contract b/w client & service provider is an interface.

Summary.

Any service requirement specification or contract b/w client & service provider or 100% pure abstract class is interface.

Chapter 8: Package

- Package

Package

It is a collection of classes, interfaces and enums.

class: File
interface: File
enum: File
package: directory/folder

Benefits: Classification

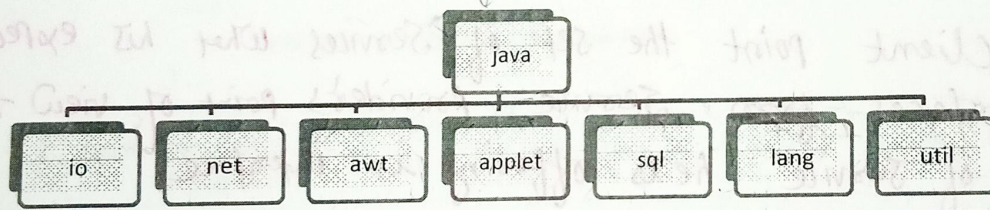
- Easier Naming
- Easier Access (Read/Write)

Types of Packages

1. Pre-Defined (Java API)
2. User-Defined

Note: Packages make use of lower case

Pre-Defined (Java API)



To call classes or interfaces from a package "import" statement is used.

1. `import <package>. *;`

This will import all classes and interfaces from the specified package.

2. `import <package>.<ClassName>;`

This will import specified class from specified package.

3. `import <package>.<InterfaceName>;`

This will import specified interface from specified package.

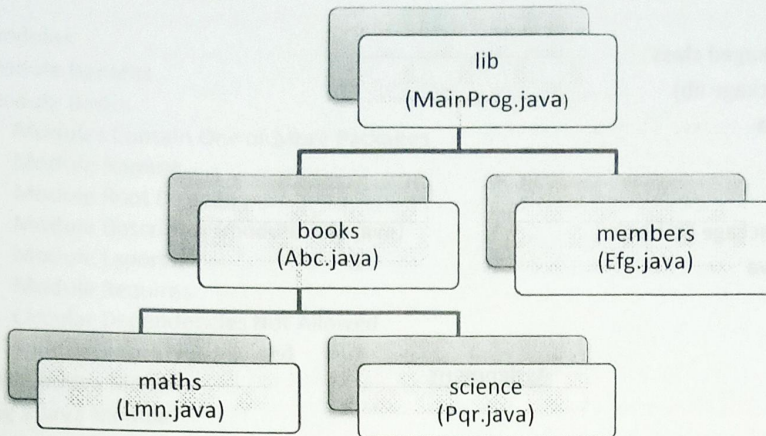
4. `import <package>.<Enum>;`

This will import specified interface from specified package.

NOTE:

- Whenever we import a package, it does not import the contents of its sub package
- By default contents of **java.lang** package is imported in every program. Classes from **java.lang** package include System, String, Wrappers Classes, Math, Object class.

Example: User Defined Packages



1. Create a folder called "lib". Within "lib" folder create two sub-folders "books" and "members", again within "books", create two sub-folders "maths" and "science".

2. Create 4 packaged classes Abc.java, Efg.java, Lmn.java

package lib.books;

```

public class Abc {
    public Abc() {
        System.out.println("This is Abc");
    }
}
  
```

//lib\books\Abc.java

those classes which are in packages are called package class

package lib.books.maths;

```

public class Lmn {
    public Lmn() {
        System.out.println("This is Lmn");
    }
}
  
```

//lib\books\maths\Lmn.java

3. Create a class called "MainProg" within "lib"

package lib;

```

import lib.books.*;
import lib.books.maths.Lmn;
import lib.books.science.Pqr;
import lib.members.Efg;
  
```

```

public class MainProg {
    public static void main(String args[])
    {
        Abc a = new Abc();
        Efg b = new Efg();
        Lmn c = new Lmn();
        Pqr d = new Pqr();
    }
}
  
```

// lib\MainProg.java

NOTE: Package classes cannot be compiled and interpreted from within the package

To compile and interpret a packaged class

Location: lib\MainProg.java (package lib)

D:\Data>javac lib\MainProg.java

D:\Data>java lib.MainProg

Location: lib\books\Abc.java (package lib.books)

D:\Data>javac lib\books\Abc.java

D:\Data>java lib.books.Abc

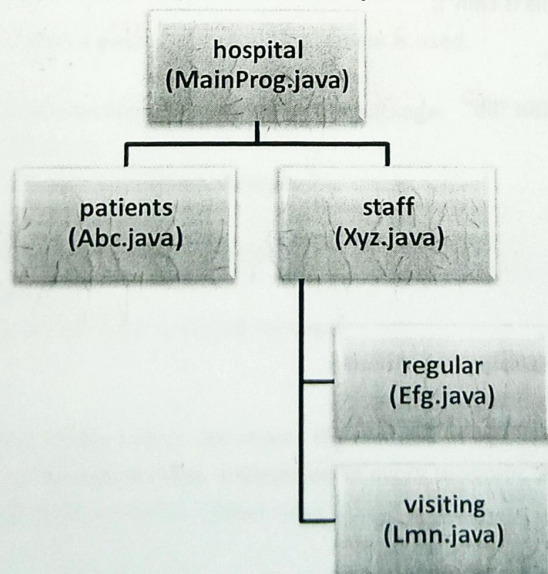
Assignment

Theory Questions

1. What are packages and why they are used? What are the various types of packages?
2. Give names of some of the pre-defined packages and which is the default package that gets imported in every program?
3. Why we use import statement? What are its variants?
4. How are packaged classes compiled and interpreted.

Practical Questions

1. Create a class called **Abc** within a package called **mypack**, create another class called **Xyz** within the same package. Create main function within class **Xyz**, in which an object of **Abc** class is to be created. Compile and interpret it.
2. Create the following package hierarchy and perform the following operations:
 - a. Write main function within **MainProg** that creates object of **Abc**, **Xyz**, **Efg**, and **Lmn**. Compile and Interpret it.
 - b. Write main functions within **Xyz** that created objects of **Lmn** and **Efg**. Compiles and Interpret it.



Chapter 9: Modules

- Java Modules
- Java Module Benefits
- Java Module Basics
 - Modules Contain One or More Packages
 - Module Naming
 - Module Root Directory
 - Module Descriptor (module-info.java)
 - Module Exports
 - Module Requires
 - Circular Dependencies Not Allowed
 - Split Packages Not Allowed
- Compiling a Java Module
- Running a Java Module
- Building a Java Module JAR File
 - Setting the JAR Main Class
- Running a Java Module From a JAR
 - Running a Java Module From a JAR With a Main Class Set
- Packing a Java Module as a Standalone Application
 - Running the Standalone Application
- Unnamed Module
- Automatic Modules
- Practical Example

Java Modules

A Java Module is a mechanism to package up your Java application and Java packages into Java modules. A Java module can specify which of the Java packages it contains that should be visible to other Java modules using this module. A Java module must also specify which other Java modules are required to do its job.

Java module is a new feature in Java 9 via the Java Platform Module System (JPMS). The Java Platform Module System is also sometimes referred to as Java Jigsaw or Project Jigsaw depending on where you read. Jigsaw was the internally used project name during development. Later Jigsaw changed name to Java Platform Module System.

Java Module Benefits

- Smaller Application Distributable via the Modular Java Platform
- Encapsulation of Internal Packages
- Start-up Detection of Missing Modules

Java Module Basics

Modules Contain One or More Packages

A Java module is one or more Java packages that belong together. A module could be either a full Java application, a Java Platform API, or a third party API.

Module Naming

A Java module must be given a unique name. For instance, a valid module name could be

```
com.vedisoft.mymodule
```

Module Root Directory

From Java 9 a module must be nested under a root directory with the same name as the module. In the example above we have a directory structure for a package named "**com.vedisoft.mymodule**".

The directory structure for the above Java package contained in a Java module of the same name, would look like this:

```
com.vedisoft.mymodule/com/vedisoft/mypack
```

Module Descriptor (module-info.java)

Each Java module needs a Java module descriptor named `module-info.java` which has to be located in the corresponding module root directory. For the module root directory `src/main/java/com.vedisoft.mymodule` the path to the module's module descriptor will be `src/main/java/com.vedisoft.mymodule/module-info.java`.

The module descriptor specifies which packages a module exports, and what other modules the module requires. These details will be explained in the following sections. Here is how a basic, empty Java module descriptor looks:

```
module com.vedisoft.mymodule {  
  
}
```

Module Exports

A Java module must explicitly export all packages in the module that are to be accessible for other modules using the module. The exported packages are declared in the module descriptor. Here is how a simple export declaration looks inside a module descriptor:

```
module com.vedisoft.mymodule {  
    exports com.vedisoft.mypack;  
}
```

To export a subpackage also, you must declare it explicitly in the module descriptor, like this:

```
module com.vedisoft.mymodule {  
    exports com.vedisoft.mypack;  
    exports com.vedisoft.mypack.util;  
}
```

You do not have to export the parent package in order to export a subpackage. The following module descriptor exports statement is perfectly valid:

```
module com.vedisoft.mymodule {  
    exports com.vedisoft.mypack.util;  
}
```

This example only exports the `com.vedisoft.mymodule.util` package, and not the `com.vedisoft.mymodule` package

Module Requires

If a Java module requires another module to do its work, that other module must be specified in the module descriptor too. Here is an example of a Java module requires declaration:

```
module com.vedisoft.mymodule {
    requires java.desktop;
}
```

This example module descriptor declares that it requires the standard Java module named java.desktop.

Circular Dependencies Not Allowed

It is not allowed to have circular dependencies between modules. In other words, If module A requires module B, then module B cannot also require module A. The module dependency graph must be an acyclic graph.

Split Packages Not Allowed

The same Java package can only be exported by a single Java module at runtime. In other words, you cannot have two (or more) modules that export the same package in use at the same time. The Java VM will complain at startup if you do.

Having two modules export the same package is also sometimes referred to as a split package. By split package is meant that the total content (classes) of the package is split between multiple modules. This is not allowed.

Compiling a Java Module

destination
module को जहाँ से सहे
destination
Module
`javac -d out --module-source-path . --module com.vedisoft.mymodule`

When javac compiles the module it writes the compiled result into the directory specified after the -d argument to the javac command. Inside that directory you will find a directory with the name of the module, and inside that directory you will find the compiled classes plus a compiled version of the module-info.java module descriptor named module-info.class.

The --module-source-path should point to the source root directory, not the module root directory. The source root directory is normally one level up from the module root directory.

The --module argument specifies which Java module to compile. In the example above it is the module named com.vedisoft.mymodule. You can specify multiple modules to be compiled by separating the module names with a comma. For instance:

... --module com.vedisoft.mymodule1, com.vedisoft.mymodule2

Running a Java Module

In order to run the main class of a Java module you use the java command, like this:

module
मोड्यूल
`java --module-path out --module com.vedisoft.mymodule/com.vedisoft.mypack.Main`

The `--module-path` argument points to the root directory where all the compiled modules are located. Remember, this is one level above the module root directory.

The `--module` argument tells what module + main class to run. In the example the module name is the `com.vedisoft.mymodule` part and the main class name is the `com.vedisoft.mypack.Main`. Notice how the module name and main class name are separated by a slash (/) character

Building a Java Module JAR File

The `jar` command needed to generate a JAR file from a compiled Java module

`jar -c -file=out/com-vedisoft-mymodule.jar -C out/com.vedisoft.mymodule .`

Setting the JAR Main Class

You can still set the JAR main class when generating the module JAR file. You do so by providing a `--main-class` argument. Here is an example of setting the main class of a Java module JAR file:

`jar -c --file=out/com-vedisoft-mymodule.jar --main-class=com.vedisoft.mypack.Main -C out/com.vedisoft.mymodule .`

Running a Java Module From a JAR

Once you have packaged your Java module into a JAR file, you can run it just like running a normal module. Just include the module JAR file on the module path. Here is how you run the main class from a Java module JAR file:

`java --module-path out -m com.vedisoft.mymodule/com.vedisoft.mypack.Main`

Running a Java Module From a JAR With a Main Class Set

If the Java module JAR file has a main class set, you can run the Java module main class with a little shorter command line. Here is an example of running a Java module from a JAR file with a main class set:

`java -jar out/com-vedisoft-mymodule.jar`

Notice how no `--module-path` argument is set. This requires that the Java module does not use any third party modules. Otherwise you should provide a `--module-path` argument too, so the Java VM can find the third party modules your module requires.

Packing a Java Module as a Standalone Application

You can package a Java module along with all required modules (recursively) and the Java Runtime Environment into a standalone application. The user of such a standalone application does not need to have Java pre-installed to run the application, as the application comes with Java included. That is, as much of the Java platform as the application actually uses.

You package a Java module into a standalone application using the `jlink` command which comes with the Java SDK. Here is how you package a Java module with `jlink`:

`jlink --module-path "out;C:\Program Files\Java\jdk-10.0.2\jmods" --add-modules com.vedisoft.mymodule --output out-standalone`

The `--module-path` argument specifies the module paths to look for modules in. The example above sets the out directory into which we have previously compiled our module, and the `jmods` directory of the JDK installation.

The `--add-modules` argument specifies the Java modules to package into the standalone application. The example above just includes the `com.vedisoft.mymodule` module.

The `--output` argument specifies what directory to write the generated standalone Java application to. The directory must not exist already.

Running the Standalone Application

Once packaged, you can run the standalone Java application by opening a console (or terminal), change directory into the standalone application directory, and execute this command:

```
bin\java --module com.vedisoft.mymodule/com.vedisoft.mypack.Main
```

The standalone Java application contains a `bin` directory with a java executable in. This java executable is used to run the application.

The `--module` argument specifies which module plus main class to run.

Unnamed Module

From Java 9 and forward, all Java classes must be located in a module for the Java VM to use them. But what do you do with older Java libraries where you just have the compiled classes, or a JAR file?

In Java 9 you can still use the `-classpath` argument to the Java VM when running an application. On the classpath you can include all your older Java classes, just like you have done before Java 9. All classes found on the classpath will be included in what Java calls the unnamed module.

The unnamed module exports all its packages. However, the classes in the unnamed module are only readable by other classes in the unnamed module. No named module can read the classes of the unnamed module.

If a package is exported by a named module, but also found in the unnamed module, the package from the named module will be used.

All classes in the unnamed module requires all modules found on the module path. That way, all classes in the unnamed module can read all classes exported by all the Java modules found on the module path.

Automatic Modules

What if you are modularizing your own code, but your code uses a third party library which is not yet modularized? While you can include the third party library on the classpath and thus include it in the unnamed module, your own named modules cannot use it, because named modules cannot read classes from the unnamed module.

The solution is called automatic modules. An automatic module is made from a JAR file with Java classes that are not modularized, meaning the JAR file has no module descriptor. This is the case with JAR files developed with Java 8 or

earlier.

When you place an ordinary JAR file on the module path (not the classpath) the Java VM will convert it to an automatic module at runtime.

An automatic module requires all named modules on the module path. In other words, it can read all packages exported by all named modules in the module path.

If your application contains multiple automatic modules, each automatic module can read the classes of all other automatic modules.

An automatic module can read classes in the unnamed module. This is different from explicit named modules (real Java modules) which cannot read classes in the unnamed module.

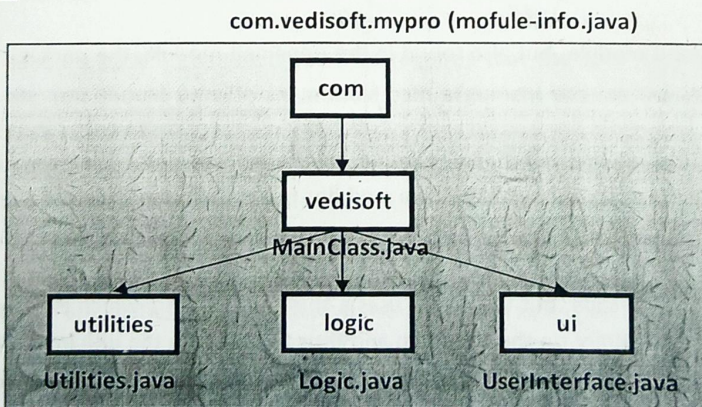
An automatic module exports all its packages, so all named modules on the module path can use the classes of an automatic module. Named modules still have to explicitly require the automatic module though.

The rule about not allowing split packages also counts for automatic modules. If multiple JAR files contain (and thus exports) the same Java package, then only one of these JAR files can be used as an automatic module.

An automatic module is a named module. The name of an automatic module is derived from the name of the JAR file. If the name of the JAR file is `com-vedisoft-mymodule.jar` the corresponding module name will be `com.vedisoft.mymodule`. The - (dash) characters are not allowed in a module name, so they are replaced with the `.` character. The `.jar` suffix is removed.

If a JAR file contains versioning in its file name, e.g. `com-vedisoft-mymodule-2.9.1.jar` then the versioning part is removed from the file name too, before the automatic module name is derived. The resulting automatic module name is thus still `com.vedisoft.mymodule`.

Practical Example



NOTE: classes from `com.vedisoft.utilities` package can also be used from outside the module

module

1. Create a folder "com.vedisoft.mypro"

2. Create the following package structure within "com.vedisoft.mypro" folder as per the above diagram.

3. Create a class called "Utilities.java" within "com.vedisoft.mypro\com\vedisoft\utilities" folder
package com.vedisoft.utilities;

```
public class Utilities {
    public Utilities() {
        System.out.println("These are utilities for the project.");
    }
}
```

4. Create a class called "Logic.java" within "com.vedisoft.mypro\com\vedisoft\logic" folder
package com.vedisoft.logic;

```
public class Logic {
    public Logic() {
        System.out.println("These are logics for the project.");
    }
}
```

5. Create a class called "UserInterface.java" within "com.vedisoft.mypro\com\vedisoft\ui" folder
package com.vedisoft.ui;

```
public class UserInterface {
    public UserInterface() {
        System.out.println("These are user interfaces for the project.");
    }
}
```

6. Create the "MainClass" within "com.vedisoft.mypro\com\vedisoft" that contains a main() and it creates objects of UserInterface class, Utilities class and Logic class. Compile and interpret it.

```
package com.vedisoft;

import com.vedisoft.utilities.*;
import com.vedisoft.ui.*;
import com.vedisoft.logic.*;

public class MainClass {

    public static void main(String args[]) {
        Utilities a = new Utilities();
    }
}
```

```
UserInterface b = new UserInterface();
Logic c = new Logic();
```

```
}
```

```
}
```

7. Create "module-info.java" within "com.vedisoft.mypro" folder

```
module com.vedisoft.mypro {
    exports com.vedisoft.utilities;
}
```

8. Compile and run the module "com.vedisoft.mypro"

```
G:\Data> javac -d out --module-source-path . --module com.vedisoft.mypro
```

```
G:\Data> java --module-path out --module com.vedisoft.mypro/com.vedisoft.MainClass
```

9. Create the Module Jar File and set "MainClass" class as the main class

```
G:\Data> jar -c --file=out-standalone/com-vedisoft-mypro.jar --main-class=com.vedisoft.MainClass -C
out/com.vedisoft.mypro .
```

NOTE: create a folder by the name "out-standalone" before executing this command .

10. Execute the Java Module from a jar

```
java -jar out-standalone/com-vedisoft-mypro.jar
```

jar file को ऐसे चले है 3रा/ command

runtime

* for every thread JVM will create a ↑ stack. Each and Every method called by and performed by that thread will be stored in the corresponding stack. after completing every method call, now corresponding entry will be removed from the stack and after empty stack, JVM will destroy by that stack.

⇓

this is called Runtime stack mechanism.